

STACK

Array & Linked List Implementations

Data Structures Lab • Insertion, Deletion, Search & Real-World Use

TOP →



What We'll Cover

A roadmap for today's lab

1

What is a Stack?

LIFO principle with a real-life analogy

2

Array Implementation

Push, Pop, Peek & Search using a fixed array

3

Linked List Implementation

Push, Pop, Peek & Search using nodes

4

Comparison

Array vs Linked List — trade-offs

5

Real-World Application

Balanced parentheses checker in C++

LIFO = Last In, First Out

Plate 3 (pushed last)

Plate 2

Plate 1 (pushed first)

Like a stack of cafeteria plates — you take from (and add to) the top only.

What is a Stack?

Core concept before we implement it

A Stack is a linear data structure that follows the **LIFO (Last In, First Out)** principle.

The element inserted last is the first one to be removed.

Push

Insert an element on top of the stack

Pop

Remove the element from the top

Peek / Top

View the top element without removing it

isEmpty

Check whether the stack has any elements

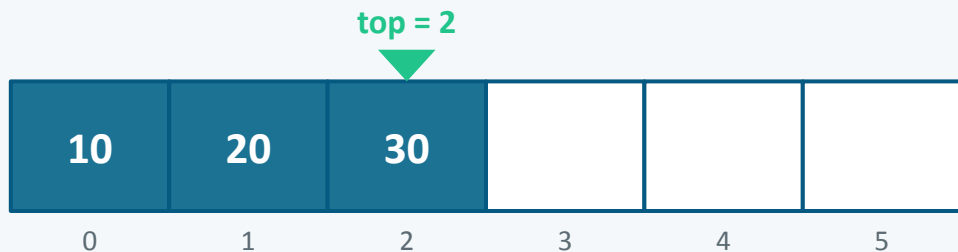
Where Stacks Are Used

- Undo/Redo in text editors
- Browser back button history
- Function call stack (recursion)
- Matching brackets / expression parsing
- Backtracking (maze, Sudoku solvers)

Array Implementation — Idea

Fixed-size contiguous memory, tracked with a 'top' index

A variable `top` always points to the index of the most recently pushed element. Push increases top; Pop decreases it.



```
array[MAX] = {10, 20, 30, _, _, _}    top = 2 (0-indexed)
```

Memory

Fixed size decided at declaration
(MAX_SIZE)

Access

$O(1)$ push/pop at the end via index

Limitation

Stack Overflow if full, wasted space
if underused

Array Stack — Push & Pop

C++ implementation

```
#define MAX 100
int arr[MAX];
int top = -1;           // empty stack
void push(int x) {
    if (top == MAX - 1) {
        cout << "Stack Overflow\n";
        return;
    }
    arr[++top] = x;      // increment top, then
    insert
}
void pop() {
    if (top == -1) {
        cout << "Stack Underflow\n";
        return;
    }
    cout << "Popped: " << arr[top] << "\n";
    top--;              // just move pointer back
}
```

Why it's $O(1)$

- push(): put value at top+1, no shifting needed
- pop(): just move the top pointer down — value stays in memory but is logically removed
- Always check bounds first: full (Overflow) or empty (Underflow)

Example: push(10) → push(20) → push(30)
→ pop()
Output: "Popped: 30" → stack now holds
[10, 20]

Array Stack — Searching for a Value

Finding a particular number

```
// Returns position from top (1-based), or -1 if
not found
int search(int x) {
    for (int i = top; i >= 0; i--) {
        if (arr[i] == x)
            return (top - i) + 1; // distance
from top
    }
    return -1; // not found
}

// Usage
int pos = search(20);
if (pos != -1)
    cout << "20 is " << pos << " from the top\n";
else
    cout << "Not found\n";
```

How it works

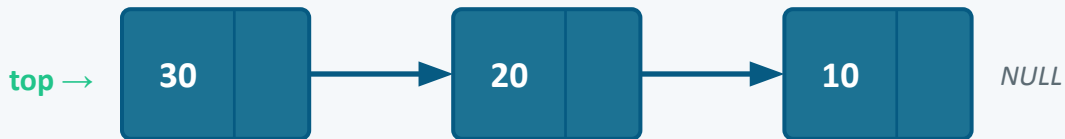
We scan from top down to index 0, since a stack only lets us reason about elements relative to the top. There is no shortcut — search is always $O(n)$ for both array and linked list versions, since a plain stack has no random access.

Stack (top → bottom): 30, 20, 10
search(20) checks 30 first, then 20 → match!
Returns 2 (2nd from the top)

Linked List Implementation — Idea

Dynamic size using nodes; 'top' is just the head pointer

Each node stores a value and a pointer to the next node. The stack's top is simply the head of the linked list — no fixed size, no overflow (until memory runs out).



Memory

Grows and shrinks dynamically — one node per push

Access

$O(1)$ push/pop at head, no wasted pre-allocated space

Trade-off

Extra memory per node for the pointer; needs new/delete

Linked List Stack — Push & Pop

C++ implementation using a Node struct

```
struct Node {
    int data;
    Node* next;
};

Node* top = NULL;           // empty stack

void push(int x) {
    Node* n = new Node();
    n->data = x;
    n->next = top;           // link new node to old top
    top = n;                 // new node becomes top
}

void pop() {
    if (top == NULL) {
        cout << "Stack Underflow\n";
        return;
    }
    Node* temp = top;
    cout << "Popped: " << top->data << "\n";
    top = top->next;         // move top to next node
    delete temp;            // free old top's memory
}
```

Why it's $O(1)$

- `push()`: create a node, point it to the old top, then re-point top — no traversal
- `pop()`: read top's data, move top to `top->next`, delete the old node
- delete temp prevents memory leaks — always free what you push

Example: `push(10) → push(20) → push(30) → pop()`
Output: "Popped: 30" → top now points to node 20

Linked List Stack — Searching for a Value

Traversing node by node from the top

```
// Returns position from top (1-based), or -1 if
not found
int search(int x) {
    Node* temp = top;
    int pos = 1;
    while (temp != NULL) {
        if (temp->data == x)
            return pos;
        temp = temp->next;
        pos++;
    }
    return -1;                // not found
}

// Usage
int pos = search(20);
if (pos != -1)
    cout << "20 is " << pos << " from the top\n";
```

How it works

Unlike an array, we cannot jump to an index — we must follow next pointers one node at a time starting from top, exactly like following a chain. This is also $O(n)$ — same complexity as the array version, but with no random access at all.

List (top → bottom): 30 → 20 → 10 → NULL
search(20) visits 30, then 20 → match!
Returns 2 (2nd from the top)

Array vs Linked List — Comparison

Choosing the right implementation

Aspect	Array-based Stack	Linked List-based Stack
Memory	Fixed size (MAX), set at compile time	Dynamic — grows/shrinks as needed
Push / Pop	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$
Overflow risk	Yes, if array is full	No (until system memory is full)
Extra memory	None (values only)	Extra pointer per node
Best when	Max size is known in advance	Size is unpredictable or large

Popular Use Case: Balanced Parentheses

Checking if brackets in an expression match correctly

Compilers use exactly this logic to check (), { }, and [] in your code. Every opening bracket is pushed; every closing bracket must match the most recent unmatched opening bracket — a perfect LIFO problem.

```
bool isBalanced(string expr) {
    stack<char> st;           // using array or
    linked-list stack
    for (char c : expr) {
        if (c=='(' || c=='{' || c=='[')
            st.push(c);
        else if (c==')' || c=='}' || c==']') {
            if (st.empty()) return false; // nothing to
            match
            char top = st.top();
            st.pop();
            if ((c==')' && top!='(') ||
                (c=='}' && top!='{') ||
                (c==']' && top!='['))
                return false;           // wrong match
        }
    }
    return st.empty();           // all opens were matched
}
```

Example: { [(1+2)] }

Push { → Push [→ Push (
On ')' pop '(' ✓ match
On ']' pop '[' ✓ match
On '}' pop '{' ✓ match
Result: Balanced ✓

Example: { [(1+2)] }

Push { → Push [→ Push (
On ']' pop '(' ✗ mismatch!
Return false immediately
Result: Not Balanced ✗